

HOW THE R3 FALCON-512 AUTHENTICATION SOLUTION WAS BUILT

A Step-by-Step Development Narrative, Rationale, and Source Log — Track A

Prepared for Omar MAHMOUD

1. Purpose of This Document

The R3 Track A report (R3_Report_Track_A_Omar_Mahmoud.docx) presents what was built and what it proved. This document is its companion: it explains why each decision was made, what it was based on, how it was actually implemented, and what problems came up along the way that the polished report does not dwell on. It is written as a chronological build log — each numbered step below happened roughly in the order presented, including the debugging and the moments where the first version of something did not work.

2. Where R1 and R2 Left Off

R1 established the problem this whole project addresses: SOME/IP Service Discovery broadcasts unauthenticated multicast messages, and the obvious fix — attaching a NIST post-quantum signature to each Offer — collides with the automotive Ethernet MTU, since post-quantum signatures are substantially larger than the ~1,500-byte frame budget. R1's original proposal was to solve this by fragmenting an ML-DSA-44 (Dilithium-2) signature across two SOME/IP-SD messages sent about 10ms apart.

R2 built the first working testbed (someipy, a Python SOME/IP library, running over Unix Domain Sockets) and did two things: it empirically proved the routing-hijack vulnerability R1 had only described in theory (a fast attacker captured the dashboard's video feed in roughly 66% of trials), and it benchmarked ML-DSA-44 against Falcon-512 directly rather than only citing published sizes. R2's official conclusion was to pivot away from ML-DSA-44 fragmentation and adopt Falcon-512 instead, because Falcon-512's ~655-byte signature avoids the MTU collision entirely, while ML-DSA-44's 2,420-byte signature does not fit in a single frame regardless of clever engineering around it.

Before any R3 work could start, this pivot needed to be confirmed rather than assumed — which is where Step 1 below begins.

3. Step 1 — Resolving a Contradiction Before Writing Any R3 Code

Before starting R3, I reviewed everything from R1 and R2: both reports, the full R2 codebase (crypto_test.py, pqc_benchmark.py, auto_lab.py, camera_service.py, attacker_service.py, dashboard_client.py, and the DTLs benchmark files), the raw R2_DATA.txt results, and R2's own LAB_NOTES.md. This last file surfaced a real discrepancy: while R2's polished report concludes the project should pivot to Falcon-512, LAB_NOTES.md (an informal working file) had a later, unpolished draft that instead proposed continuing with ML-DSA-44 and the original fragmentation approach for another semester.

Rather than silently pick one, I flagged this contradiction directly and asked which direction R3 should take and what form the first deliverable should take. The response was to proceed with the Falcon-512 pivot and to produce an architecture/design document first. That choice is the foundation every subsequent step in this document builds on — R3 exists, in its current form, because that specific fork in the road was resolved explicitly rather than assumed.

4. Step 2 — Confirming Falcon-512 Actually Fits the Target Hardware

R2's Falcon-512 benchmark was measured on an Apple M4 laptop, which is not representative of an automotive ECU's Cortex-M-class microcontroller. Before committing R3's design to Falcon-512, I researched whether it is realistically implementable on the constrained hardware this project ultimately targets, rather than trusting a desktop-class number.

The professor's own suggestion pointed at pqm4 (Kannwischer et al.), the standard reference framework for benchmarking NIST PQC candidates on ARM Cortex-M4 microcontrollers, which confirmed Falcon-512 has an existing, working embedded implementation rather than being a theoretical fit. Checking Falcon's standardization status turned up NIST's FIPS 206 draft, which as of the design phase was circulated for public comment with a final standard expected in late 2026 or 2027 — an important caveat carried into both the design document and the final report, since it means Falcon/FN-DSA should still be described as a standard in progress.

The most consequential piece of research at this stage was a specific, known problem with Falcon on Cortex-M4-class hardware: Falcon's signing procedure relies on a Gaussian sampler that requires double-precision floating-point arithmetic to meet its formally proven security error bounds, but the Cortex-M4F core (the automotive-class chip family this project targets) only has a single-precision hardware FPU. T. Pornin's 2025 note on Falcon and ARM Cortex-M4 raised this precision concern directly, and a newly published paper from NXP Research, "TWFalcon: Triple-Word Arithmetic for Falcon" (Halmans, van Vredendaal, Schneider, Custers, and Güneysu), proposes a concrete fix — triple-word floating-point arithmetic reaching at least 72 bits of precision using only single-precision hardware, at a measured cost of about 1.84× versus full double-precision emulation. This paper demonstrates its approach directly on a NUCLEO-L4R5ZI board, which is not a coincidence: it directly informed the later hardware recommendation for R4 (see Section 18), since choosing a board this specific paper already validates removes a major open risk from the hardware phase before it even starts.

5. Step 3 — Understanding someipy's Real Architecture, Not Assuming It

A design decision this project depends on — where, exactly, a verification check should be inserted — required reading someipy's actual installed source rather than inferring its behavior from its public API or documentation. Two things emerged from this that shaped everything downstream: first, someipy is daemon-mediated — ECU scripts (the camera, dashboard, and attacker processes) never touch a raw socket themselves; they all talk to a single background process, someipyd, over Unix Domain Sockets, and it is someipyd that owns the actual multicast socket and the routing table. Second, the installed version (2.1.2) contains two

parallel internal implementations of Service Discovery logic — an older set of files under `_internal/someip_sd_*.py` and a newer package under `_internal/_sd/` — and determining which one is actually active at runtime required tracing the daemon's own imports rather than guessing from file names or modification dates.

This groundwork is what made the later architecture decisions in Steps 6 and 7 possible: without confirming that `someipyd` is the single place all routing decisions pass through, there would be no reliable way to know where a security gate could be inserted such that no ECU script could bypass it.

6. Step 4 — Designing the Wire Format

The PQ-Signature Option's byte layout reuses the generic AUTOSAR SOME/IP-SD option header convention: a 2-byte Length field, a 1-byte Type field, and a 1-byte Reserved field, where the Length field's value counts everything after itself (Type, Reserved, and payload), not including the Length field itself. This convention was confirmed two ways: against the text of the AUTOSAR SOME/IP-SD specification directly, and as a sanity check against `someipy`'s own source, where the constant `SD_IPV4ENDPOINT_OPTION_LENGTH_VALUE = 9` for the existing IPv4 Endpoint Option only makes arithmetic sense under this exact convention. Matching this convention exactly (rather than inventing a new one) is what keeps a future embedded implementation able to attach the same option directly inside the standard Options Array without a wire-format change.

7. Step 5 — Finding the Endpoint-Substitution Replay Gap

This was the single most important design decision made in R3, and it did not come from a citation — it came from deliberately attacking the first draft of the design before writing any implementation code. If a Falcon-512 signature only covers the OfferService entry fields (Service ID, Instance ID, Major Version, TTL), an attacker does not need to break Falcon-512, forge a signature, or even possess a private key at all: they only need to capture one legitimate, validly-signed Offer message from the real Camera ECU and re-broadcast it with a different IPv4 Endpoint Option pointing at their own port. The signature over the entry is still perfectly valid, because the entry itself was never modified — only the endpoint attached alongside it changes. This is a direct continuation of the exact routing-hijack vulnerability R2 already proved (Zelle et al.'s race-condition analysis), except that authentication alone, scoped incorrectly, does nothing to stop it.

Having found this gap through direct reasoning about the design rather than a source, I then checked whether the wider literature had independently reached the same conclusion, as a form of external validation before committing to the fix. Two 2023–2025 papers proposing DNSSEC/DANE-style authentication for automotive service discovery (Mueller et al., and a 2025 arXiv paper extending that work with DANCE) reach the same underlying principle from a completely different mechanism: a signature or credential must be bound to a fresh, endpoint-specific challenge, not merely to service identity. Finding this independent convergence — different authentication mechanism, same structural requirement — increased confidence that

the fix (defining the signed scope to include the endpoint and a monotonic counter, not just the service entry) was addressing a fundamental property of the problem rather than an artifact specific to this one implementation.

8. Step 6 — Replacing “Last-In-Wins” with “Last-Valid-In-Wins”

R2 had already diagnosed that the gateway's “Last-In-Wins” routing rule is the mechanical cause of the hijack — it exists deliberately, so that a backup sensor can take over from a failed primary ECU without a manual failover step, and that property is legitimate and worth keeping. The fix designed here does not remove Last-In-Wins; it constrains it to only apply among senders who can prove they hold the legitimate signing key and are presenting a fresher (higher-counter) message than the last one accepted. This is implemented as a small trust store that tracks, per Service ID and Instance ID, the highest counter seen in a validly-signed message, and only allows a routing update when both the signature verifies and the counter is strictly greater than the last accepted value — preserving genuine failover while closing both forgery and stale replay.

9. Step 7 — Choosing the Companion-Message Architecture

At the design stage, two integration paths were considered for actually attaching the PQ-Signature Option to real traffic: patch someipy's native Options Array serialization and type enum directly, or carry the Falcon-512 signature as a separate companion UDP multicast message alongside the unmodified native OfferService broadcast. Track A implements the second option, and this was a deliberate engineering tradeoff rather than a simplification for its own sake: patching someipy's internal enum and serialization code would mean modifying logic deep inside a third-party library, increasing the risk of subtly breaking its existing, working Service Discovery behavior for a project that ultimately wants to keep that behavior AUTOSAR-compliant and unchanged. The companion message's payload is still structured as exactly the PQ-Signature Option TLV defined in Step 4, so it remains byte-compatible with a future native implementation — the companion transport is a Track A implementation detail, not a change to the underlying wire format design.

10. Step 8 — Implementation, File by File

With the design settled, implementation proceeded in the following order, each file building on the one before it:

- `pq_crypto.py` — the cryptographic core: key generation, signing, and verification wrapping the `pqcrypto` library's Falcon-512 bindings; the signed-scope construction (packing Service ID, Instance ID, Major Version, TTL, endpoint IP/port, and counter into the exact bytes that get signed, per Step 7's decision); and the `PQSignatureOption` and `CompanionAuthMessage` classes that serialize and parse the wire format from Step 4.
- `trust_store.py` — the `LastValidState` and `TrustStore` classes implementing Step 6's rule: per-service public keys, and per-(service, instance) last-accepted counter and endpoint, with load/save to a JSON file so the gateway's trust state survives a restart.

- `setup_keys.py` — a one-time script generating the Camera ECU's real Falcon-512 keypair and writing the initial trust store, standing in for the manufacturing-time key provisioning process a real vehicle would use.
- `secure_daemon.py` — the gateway integration itself: a launcher that imports `someipy`'s daemon module and patches two of its methods at runtime (rather than editing the installed package), implementing the OfferService gate from Step 7 and, later, the second gate described in Step 11, plus the `asyncio` task that listens for and verifies companion messages.
- `camera_service.py` — modified from R2 to sign its scope once per boot session (not once per message — Falcon-512 signing is comparatively expensive, so this amortizes that cost off the safety-critical path) and to periodically re-announce that same already-signed companion message, mirroring how OfferService itself is already cyclically re-broadcast.
- `attacker_service.py` — extended from R2 with a new attack mode implementing the exact endpoint-substitution replay from Step 5: it sniffs the multicast group for a real, validly-signed companion message, then rebroadcasts a mutated copy with the endpoint fields swapped to its own port while re-using the stolen signature bytes verbatim, alongside R2's original unmodified flood mode.
- `dashboard_client.py` and `run_r3_experiments.py` — the measurement and orchestration scripts: the former measures Time-to-First-Frame and logs the malicious-frame ratio exactly as R2 did, and the latter is the script meant to be run directly on Omar's own machine to reproduce every result in the R3 report end to end.

11. Step 9 — The Debugging Journey

Several problems surfaced only once the system was actually run, rather than during design, and are documented here because they are useful context for reproducing the results, even though none of them are security findings in their own right:

- Sending to the Service Discovery multicast group failed with “Network is unreachable” in the sandboxed test environment, because that environment has no default network route — only loopback. The fix was to explicitly set the `IP_MULTICAST_IF` socket option to the loopback interface before sending, in both `camera_service.py` and `attacker_service.py`.
- Joining the multicast group to sniff for companion messages failed with “No such device” when using the conventional “any interface” address (0.0.0.0) for `IP_ADD_MEMBERSHIP`; this required an explicit interface address instead.
- Long-running background processes (the daemon, camera, and attacker) could not be reliably managed using ordinary shell backgrounding in the sandboxed execution environment used to validate this project — any process left detached at the end of a command was silently killed. The reliable pattern that emerged was to launch and manage every process from within a single controlling Python script using `subprocess.Popen`, with an explicit kill-and-wait sequence before the script exits.

None of these are properties of a real vehicle network; they are specific to validating this project inside a sandboxed environment, and are called out so they are not mistaken for a design property if the same commands are re-run somewhere with normal networking.

12. Step 10 — Finding a Pre-Existing Bug in the R2 Codebase

While writing `pq_crypto.py`'s verification path, I tested the underlying `pqcrypto` library's function signature directly instead of assuming R2's existing usage of it was correct, since verification code is exactly the kind of logic that fails silently if it is wrong. This turned up a real bug: `pqcrypto.sign.falcon_512.verify` and `ml_dsa_44.verify` both expect the argument order `verify(public_key, message, signature)`, but R2's `crypto_test.py` and `pqc_benchmark.py` both call `verify(public_key, signature, message)`, with the last two arguments swapped. This does not raise an exception — it silently returns `False` on every single call. The practical consequence is that every “verification” timing number reported anywhere in the R1/R2 material was timing a call that always failed, not a genuine cryptographic verification. This was fixed in all of R3's new code and flagged explicitly, both here and in the R3 report, so the affected R2 benchmarks can be corrected and re-measured.

13. Step 11 — Finding the `SendEventRequest` Gating Gap

This was the most important empirical finding of the implementation phase, and it was discovered by the results themselves contradicting expectations, not by additional design review. After implementing the `OfferService` gate from Step 7 (rejecting any `Offer` whose endpoint does not match the trust store), the very first end-to-end flood-attack test against the fully gated gateway still showed a 65.0% malicious-frame ratio — matching R2's original ~66% unauthenticated baseline almost exactly, as if the gate were not active at all.

Investigating why led back into `someipy`'s source a second time (building directly on the architectural groundwork from Step 3), and revealed that the library's internal `_handle_send_event_request` method forwards an event to every current subscriber of a given (Service ID, Instance ID, EventGroup ID) the moment any locally connected client calls `send_event()` for it — without ever checking whether that client's claimed source endpoint matches whichever `OfferService` was actually accepted into the routing table. In effect, `someipy` 2.1.2 has two separate authorization surfaces that need to be gated independently: the Service Discovery / routing-table path (patched in Step 7), and the local event-forwarding path, which is a completely different code path that the `OfferService` gate never touches. A rogue ECU can inject a malicious frame straight into existing subscribers over the daemon's local socket API without its own `OfferService` ever being accepted at all.

The fix was a second monkeypatch, applied to `_handle_send_event_request`, checking the claimed source endpoint of every `send_event()` call against the trust store's currently-trusted endpoint before forwarding it. Adding this second gate brought the malicious-frame ratio down to 0.0% across every attacker delay and both attack modes reported in the R3 report. This finding is presented as a genuine, non-obvious contribution of this phase: it shows that securing SOME/IP Service Discovery is necessary but not sufficient in this particular library's architecture, and that

any future work built on someipy needs to treat event-forwarding authorization as a distinct concern from routing-table authenticity.

14. Step 12 — Running the Real Experiments

With both gates in place, the exact experimental methodology from R2 was re-run rather than redesigned, to keep results directly comparable: the same Service ID (0x1234), Instance ID (0x0001), EventGroup ID (0x4000), and Event ID (0x8000); ten baseline Time-to-First-Frame runs; the flood attack at 1000ms, 100ms, and 10ms attacker delays; and the new endpoint-substitution replay attack at 100ms and 10ms delays. Every number in the R3 report — the 653–686-byte on-wire sizes, the ten individual latency measurements, the 66%-to-0% attack ratios, and the attacker/gateway log excerpts confirming a real stolen signature was rejected — was produced by actually executing this code end to end, not by estimating or projecting expected behavior.

15. Step 13 — Building the R3 Report

The five figures in the R3 report (the sequence diagram, the wire-format diagram, the gateway architecture diagram, the latency chart, and the attack-ratio chart) were generated programmatically with matplotlib directly from the real data captured in Step 12, rather than drawn by hand, so the charts cannot drift from the underlying numbers. The report document itself was built to match R1 and R2's exact visual style rather than approximate it: the actual R2 .docx file was inspected directly (its underlying XML) to extract the precise fonts, sizes, and formatting used — Calibri throughout, a 14-point bold centered title, 12-point bold black section headings (not Word's default colored heading styles), and 12-point justified body text with a first-line indent — and every one of those values was matched exactly in the R3 report's generation script. The finished document was then converted to PDF and rendered page by page for visual inspection before being delivered, the same verification step used for this document.

16. What This Solution Solves

- Closes the routing-hijack vulnerability R2 proved empirically: the malicious-frame ratio drops from 66% to 0% across every attacker speed tested.
- Closes the endpoint-substitution replay gap identified in Step 5, which is a materially different attack from a naive forged Offer: an attacker with a genuinely stolen, validly-signed message still cannot redirect it to a different endpoint.
- Avoids the MTU collision R1 first identified and R2 confirmed by measurement, without needing R1's original fragmentation approach at all, because Falcon-512's signature is small enough to fit in a single datagram with room to spare.
- Preserves the legitimate fault-tolerant failover property of “Last-In-Wins” while removing its exploitability, rather than removing the feature outright.
- Closes a second, independent bypass path (SendEventRequest) specific to the someipy library's architecture, which would not have been found without adversarial, end-to-end testing.

- Corrects a latent, silent correctness bug in the existing R2 codebase's own cryptographic verification benchmarks.

17. How This Builds on R1 and R2

R1 defined the problem and the MTU-collision constraint, and proposed fragmenting an ML-DSA-44 signature as the first candidate solution. R2 built the first working testbed, proved the routing-hijack vulnerability was real rather than theoretical, benchmarked the two realistic algorithm candidates directly, and made the evidence-based decision to pivot to Falcon-512 because it avoids the MTU collision outright. R3 Track A is the phase in which that pivot stops being a conclusion on paper and becomes a running system: the same testbed, the same identifiers, and the same attack methodology R2 established are reused throughout, specifically so that R3's results remain directly comparable to R2's rather than starting over with a new baseline.

18. Contribution to R3 and Preview of R4

Within R3 as a whole, this document and the accompanying report cover Track A only; Track B (hardware benchmarking on physical Cortex-M4 hardware) is deferred until the two NUCLEO-L4R5ZI boards selected for this project are procured. Track A's software artifacts are deliberately structured to carry directly into that hardware phase rather than being thrown away: the signed-scope construction and the PQ-Signature Option's TLV byte layout (Step 4) are already wire-compatible with a native embedded implementation, so Track B's work is to port the Falcon-512 sign/verify routines themselves (via pqm4) onto the board and swap them into the same signed-scope logic already implemented in `pq_crypto.py`, rather than redesigning the protocol from scratch. The single-precision floating-point constraint identified in Step 2 (and NXP Research's TWFalcon fix for it) is already known and documented before Track B begins, rather than being an unpleasant surprise partway through it — which is precisely the kind of risk this document's Step 2 research was meant to retire early. Track B's results, once measured, are expected to replace the desktop-class proxy timings used throughout R2 and R3 with real embedded cycle counts, and will be the primary subject of the R4 phase that follows.

19. Sources and References Used Throughout

This section consolidates every external source that materially shaped a decision described above, organized by what it was used for rather than by citation order.

Protocol and Standards Ground Truth

AUTOSAR, "Specification of SOME/IP Service Discovery Protocol," AUTOSAR Standard, Release 22-11. — Defined the generic Options Array TLV convention (Length/Type/Reserved) reused for the PQ-Signature Option in Step 4, and is the baseline Service Discovery behavior this project authenticates without altering.

IEEE Std 802.3bw-2015 (100BASE-T1). — Source of the ~1,500-byte automotive Ethernet MTU figure used in every MTU-budget calculation across R1, R2, and R3.

National Institute of Standards and Technology (NIST), FIPS 206 (Draft): FN-DSA (Falcon). — Confirms Falcon/FN-DSA's real standardization status (draft, not yet finalized) as of the design phase — used in Step 2 to scope how confidently "production-ready" the algorithm choice could be described.

Security Literature

D. Zelle, T. Lauser, D. Kern, and C. Krauß, "Analyzing and Securing SOME/IP Automotive Services with Formal and Practical Methods," ARES 2021. — Source of the race-condition / Last-In-Wins vulnerability analysis that both R2's original hijack attack and R3's endpoint-substitution attack (Step 5) are direct continuations of.

C. Rathweg, C. Neumann, and T. Pöppelmann, "Impacts of Post-Quantum Cryptography on Automotive Security: A Case Study," ESCAR Europe 2023. — Confirmed the MTU collision problem on real automotive chips and compared PQC candidates, reinforcing the case for pivoting to Falcon-512 in Step 2.

M. Mueller, T. Häckel, P. Meyer, and F. Korf, "Authenticated and Secure Automotive Service Discovery with DNSSEC and DANE," IEEE VNC 2023. — Independent validation, from a different mechanism, of the endpoint-binding principle behind the Step 5 fix.

"Building Automotive Security on Internet Standards: An Integration of DNSSEC, DANE, and DANCE to Authenticate and Authorize In-Car Services," arXiv:2506.13261, 2025. — Further, more recent independent validation of the same endpoint-binding principle used in Step 5.

Algorithm and Hardware Feasibility

M. Kannwischer et al., "pqm4: Testing and Benchmarking NIST PQC on ARM Cortex-M4," Second PQC Standardization Conference, 2019. — The professor-recommended, standard reference framework confirming Falcon-512 has a real embedded implementation; also the basis for the NUCLEO-L4R5ZI board recommendation carried into Track B and R4.

T. Pornin, "Falcon on ARM Cortex-M4: an Update," IACR ePrint 2025/123. — Raised the double-precision floating-point / Cortex-M4F single-precision FPU mismatch that TWFalcon (below) was found to directly address.

S. Halmans, C. van Vredendaal, T. Schneider, F. Custers, and T. Güneysu, "TWFalcon: Triple-Word Arithmetic for Falcon; Giving Falcon the Precision to Fly Securely," IACR Cryptology ePrint Archive, Paper 2025/1991. — Directly informed both the confidence in Falcon-512's embedded feasibility and the specific choice of the NUCLEO-L4R5ZI board for R4, since this paper demonstrates its precision fix on that exact board.

Software Tools

chrizog, "someipy: A Python Library implementing the SOME/IP Protocol," GitHub, github.com/chrizog/someipy (v2.1.2 used in this project). — The SOME/IP simulation library this entire testbed is built on; its source was read directly (Step 3) to determine where the security gates in Steps 7 and 11 could actually be inserted.

pqcrypto (Python bindings for liboqs/PQClean Falcon-512 and ML-DSA-44 implementations).

— Used for all signing/verification operations in `pq_crypto.py`; testing its real function signature directly is what surfaced the argument-order bug described in Step 10.

matplotlib, docx (npm), and LibreOffice / pdftoppm. — Used respectively to generate the R3 report's figures from real experimental data, to build the report document itself in R1/R2's exact visual style, and to render it to images for visual verification before delivery (Step 13).